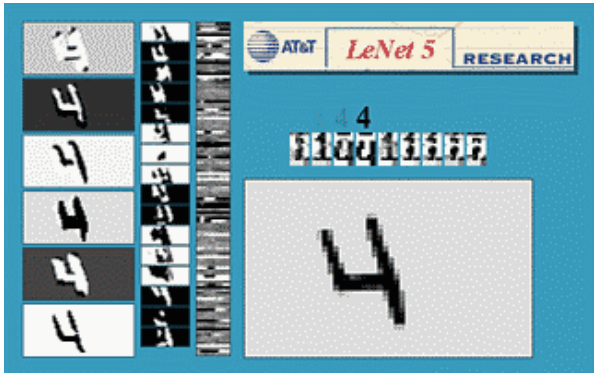
Supervised Learning: Linear and Logistic Regression

Mahesh Mohan M R
Department of AI
Indian Institute of Technology Kharagpur

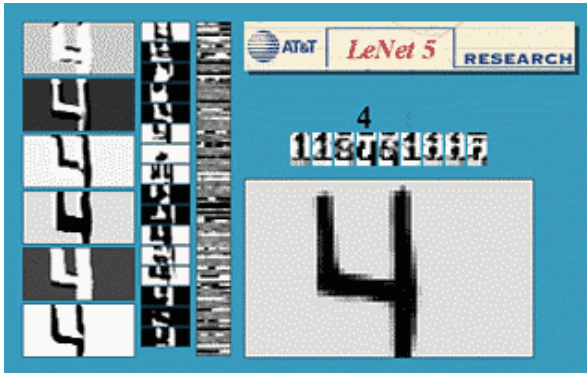
Recap



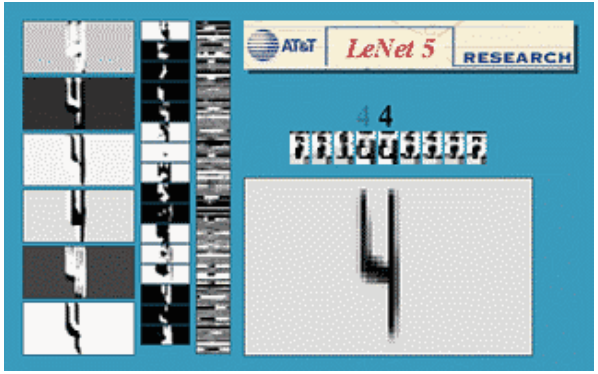
Translation invariance



Rotation invariance



Scale invariance



Squeeze invariance

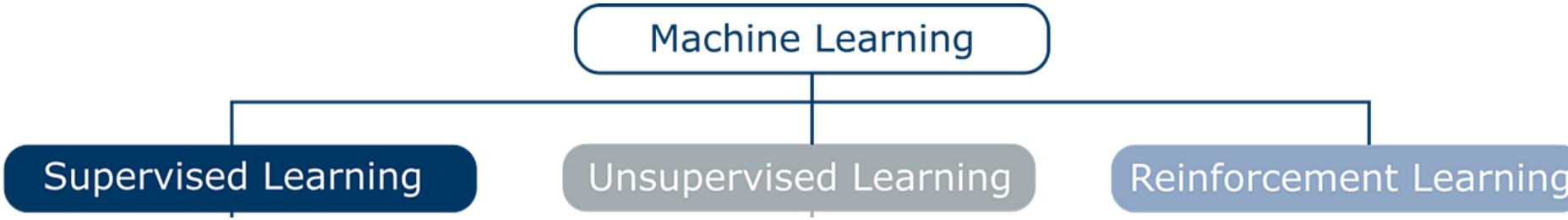


Stroke-width invariance



Noise invariance

Recap


Machine Learning
Supervised Learning
Unsupervised Learning
Reinforcement Learning

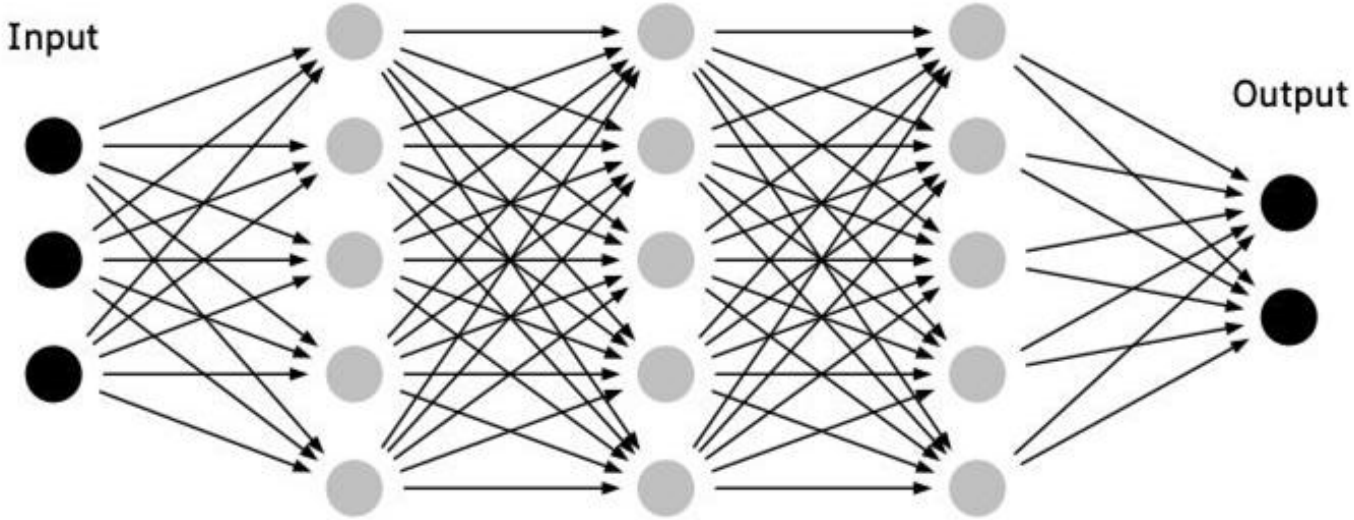

Yellow color


VS

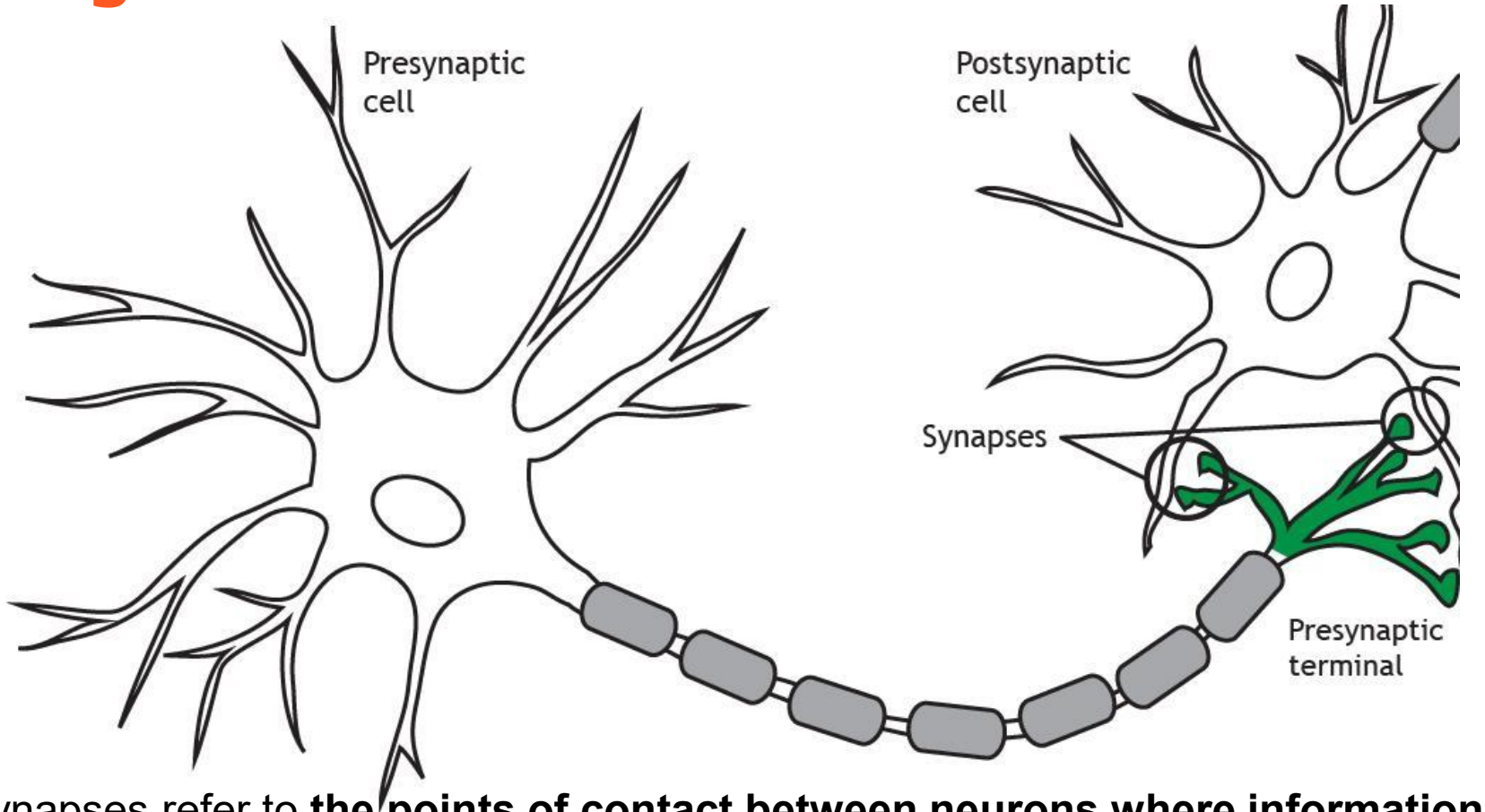


Artificial Neural Network



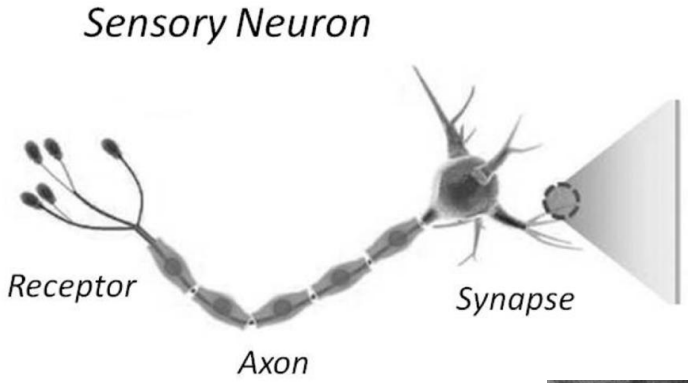


Biological Neuron



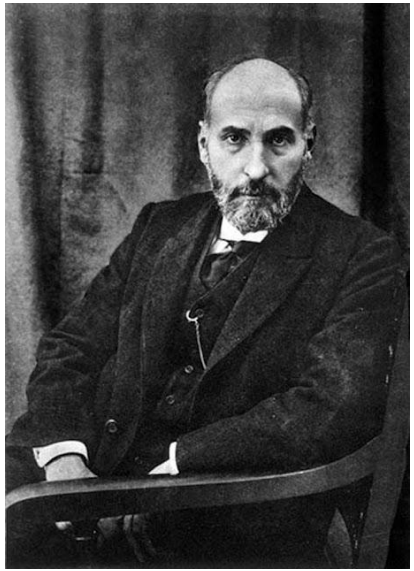
Synapses refer to **the points of contact between neurons where information is passed from one neuron to the next**.

Biological Neuron to Neural Network

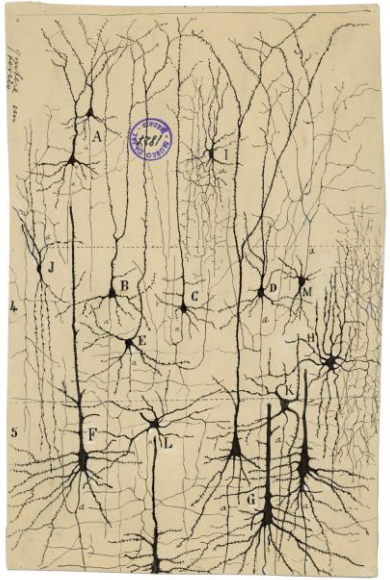


Neural Network

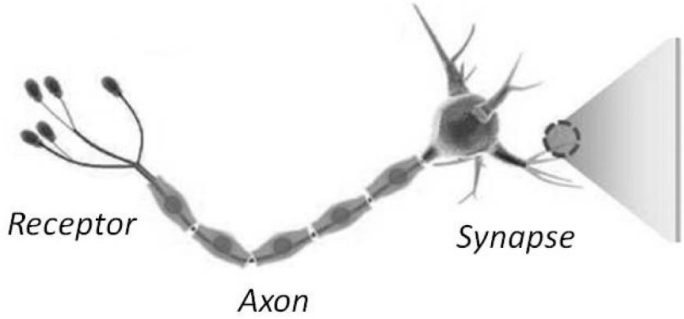




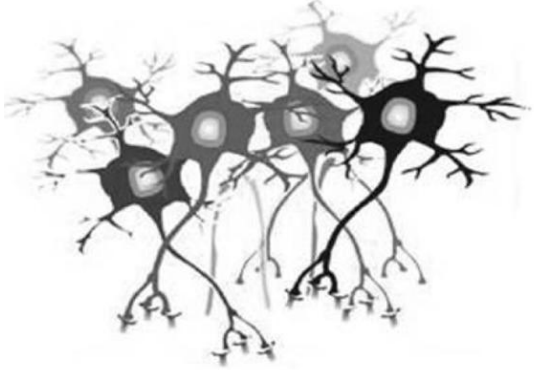
Santiago Ramón y Cajal
(1906 Nobel Laureate)

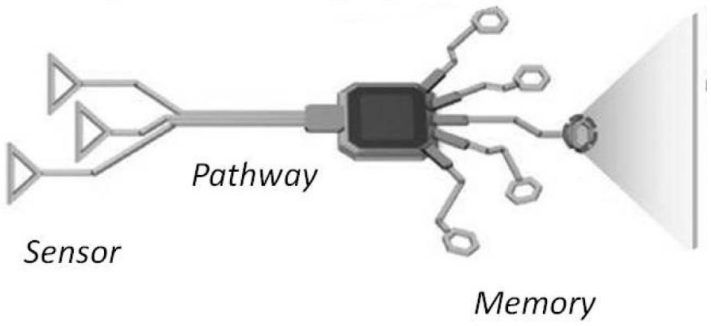


Artificial Neuron to Neural Network

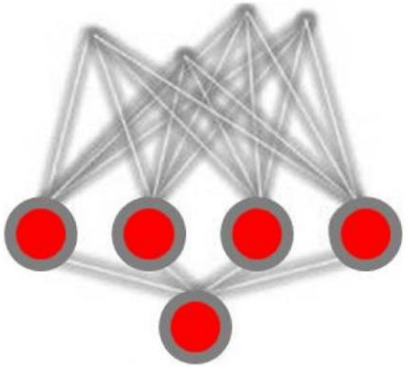


Neural Network

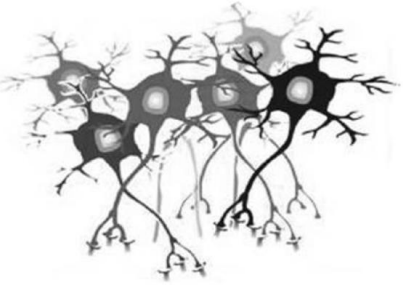


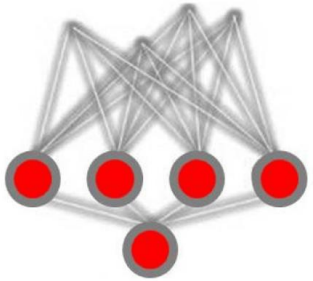


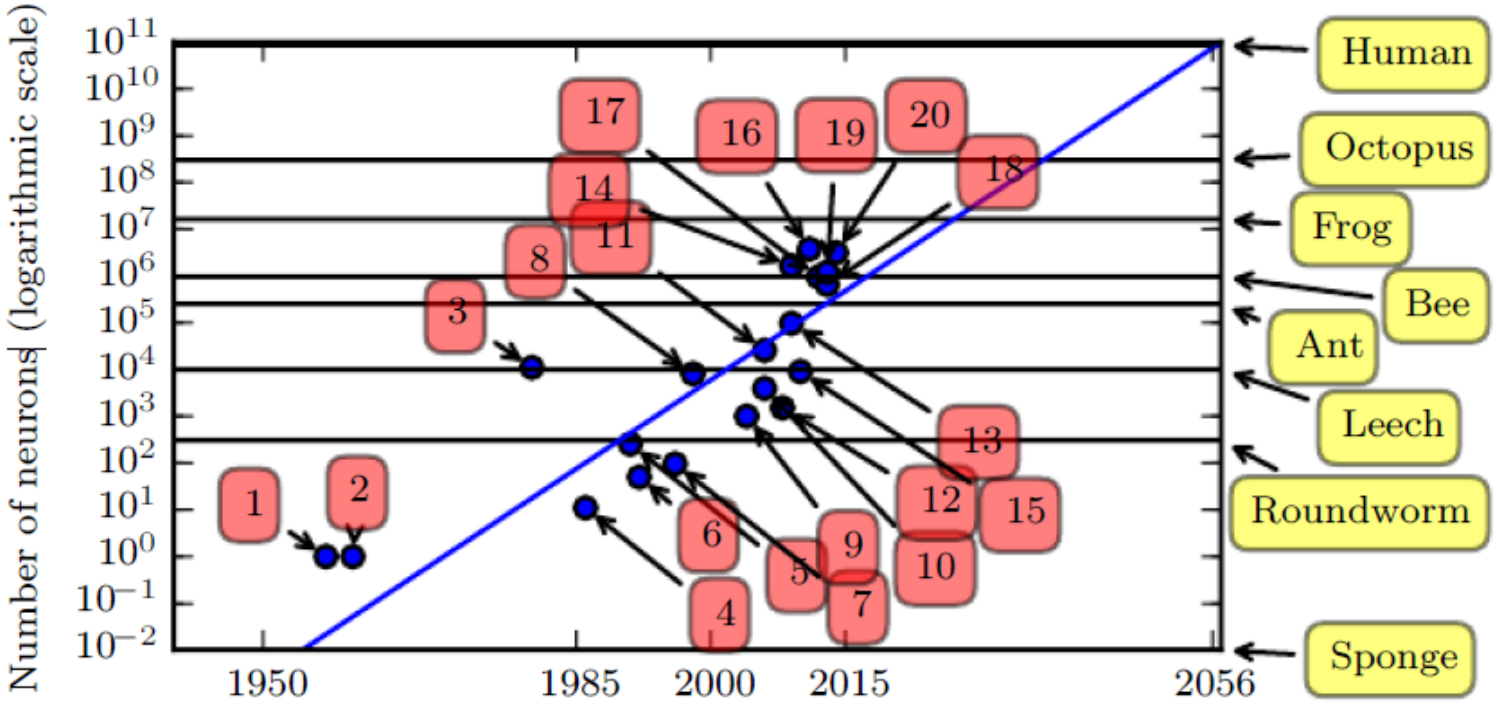
Artificial Neural Network



Biological Neural Network (NN) to Artificial NN

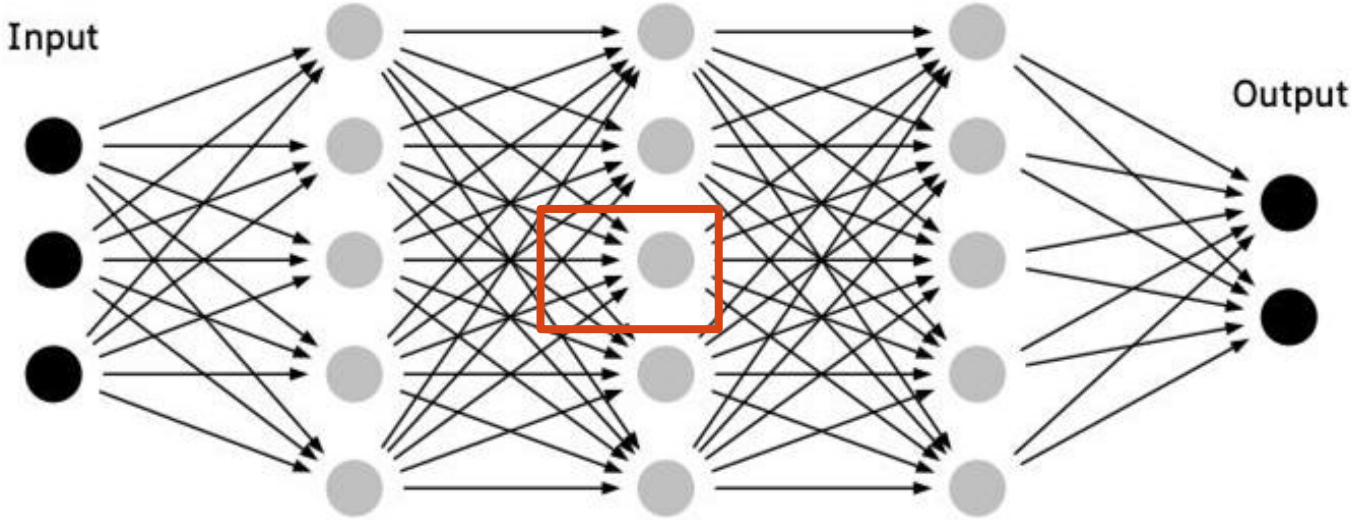






Artificial Neuron in ANN





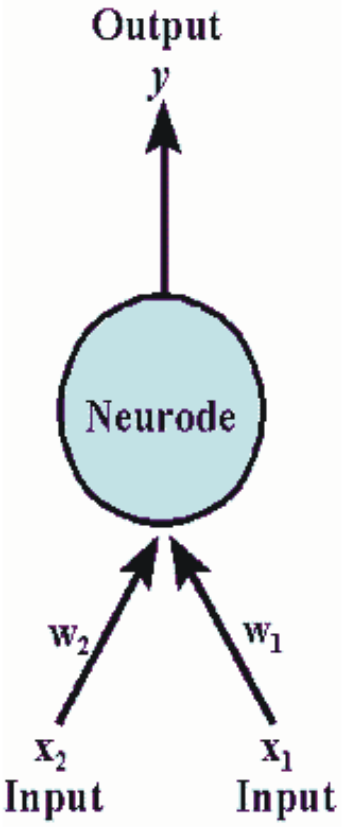
History: McCulloch Pitts Neuron - 1943



Warren McCulloch

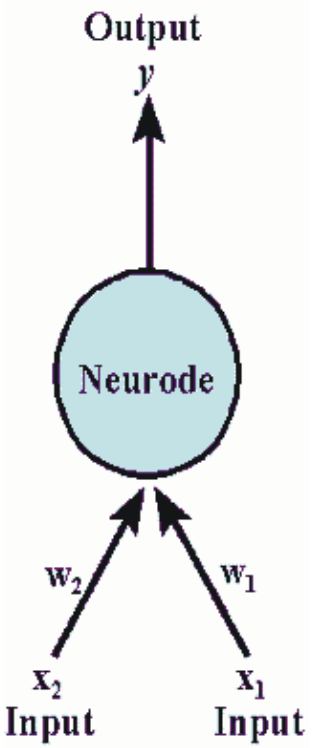


Walter Pitts



1943 paper, "A Logical Calculus of Ideas Immanent in Nervous Activity"

History: McCulloch Pitts Neuron - 1943



- Inputting either a 1 or 0 for each of the inputs, where 1 represented true and 0 false.
- Weights: Acts as a tag for excitatory (+1) or inhibitory (-1) paths
- The threshold was given a real value, say 1, which would allow for a 0 or 1 output if the threshold was met or exceeded

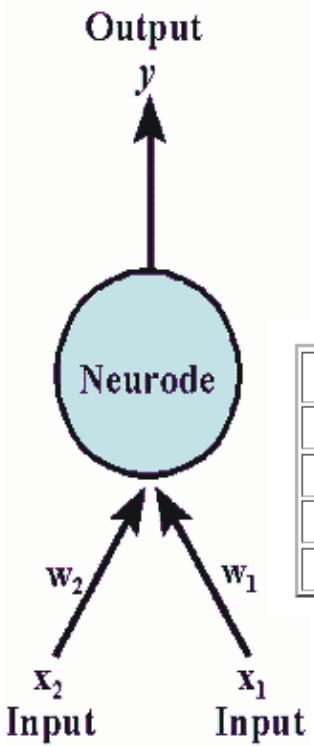
Input x_1	Input x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

$$\sum x_i w_i \geq \theta$$

1943 paper, "A Logical Calculus of Ideas Immanent in Nervous Activity"

History: McCulloch Pitts Neuron - 1943





- Inputting either a 1 or 0 for each of the inputs, where 1 represented true and 0 false.
- Weights: Acts as a tag for excitatory (+1) or inhibitory (-1) paths
- The threshold was given a real value, say 1, which would allow for a 0 or 1 output if the threshold was met or exceeded

Input x_1	Input x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

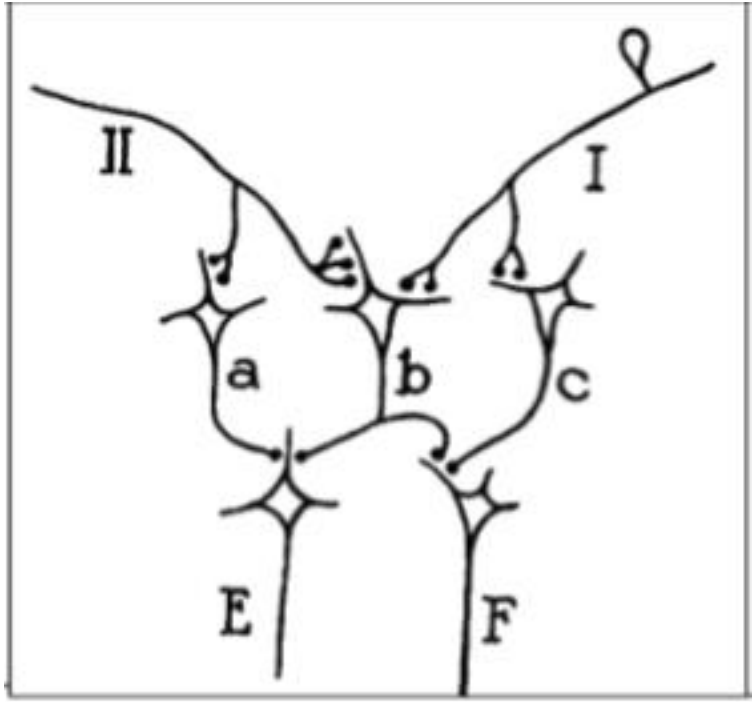
$$\sum x_i w_i \geq \theta$$

1943 paper, "A Logical Calculus of Ideas Immanent in Nervous Activity"

History: Hebbian cell - 1949



Research Area: The neurophysiology of learning and memory



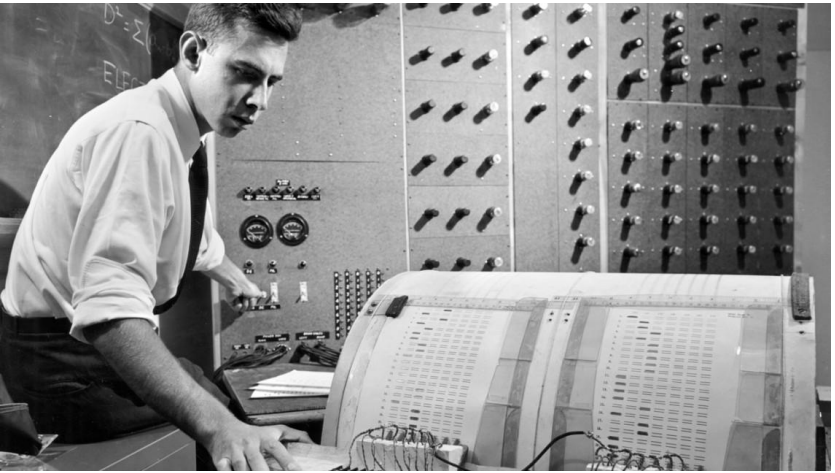
Chemical Neurotransmitters

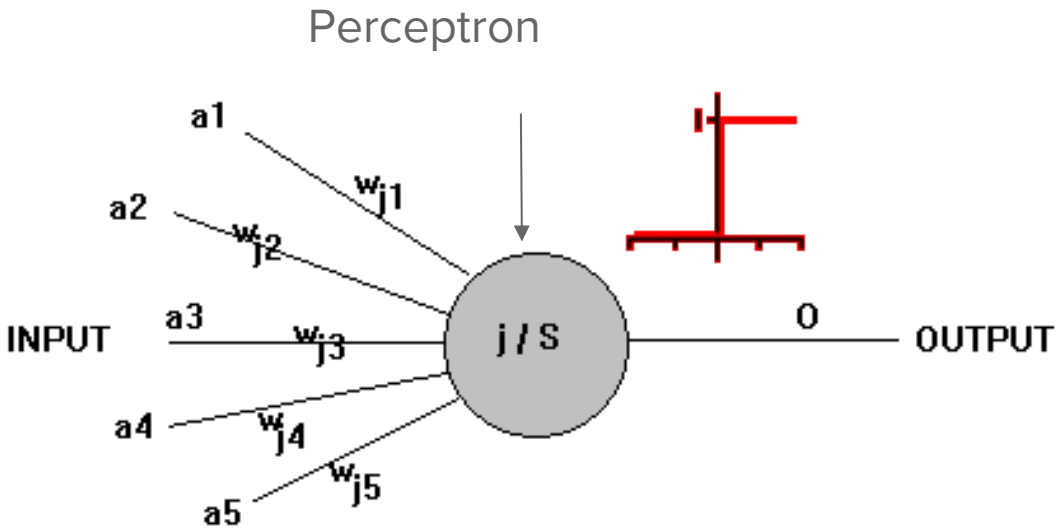
1949 book, "The Organization of Behavior".

History: Perceptron - 1958



Frank Rosenblatt, using the McCulloch-Pitts neuron & the findings of Hebb, went on to develop the first perceptron



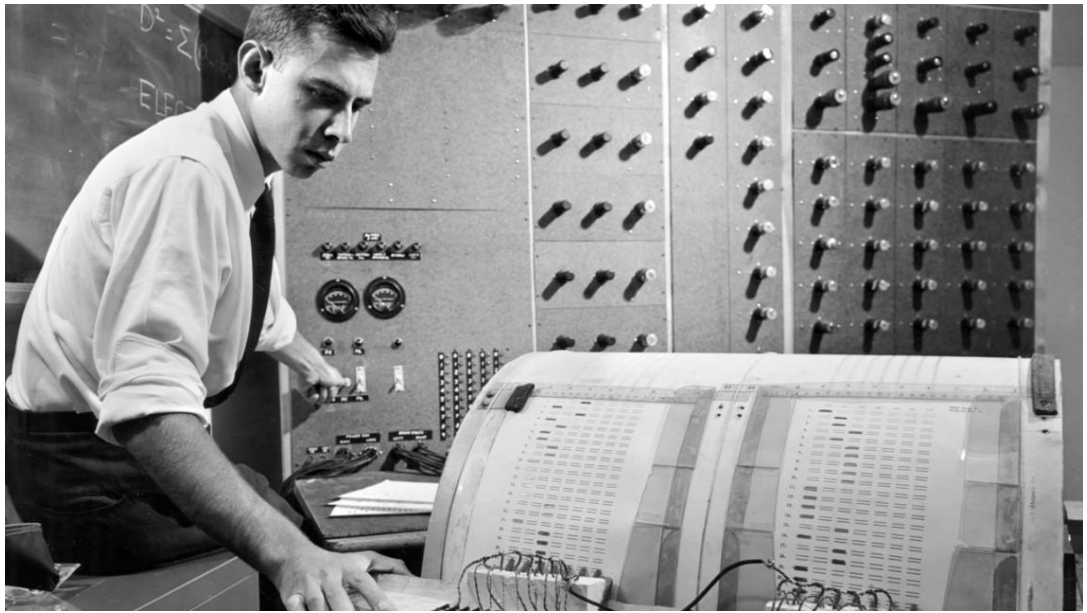


The first machine “capable of having an original idea.”

(1958). "The perceptron: A probabilistic model for information storage and organization in the brain"

History: Perceptron - 1958: First AI wave

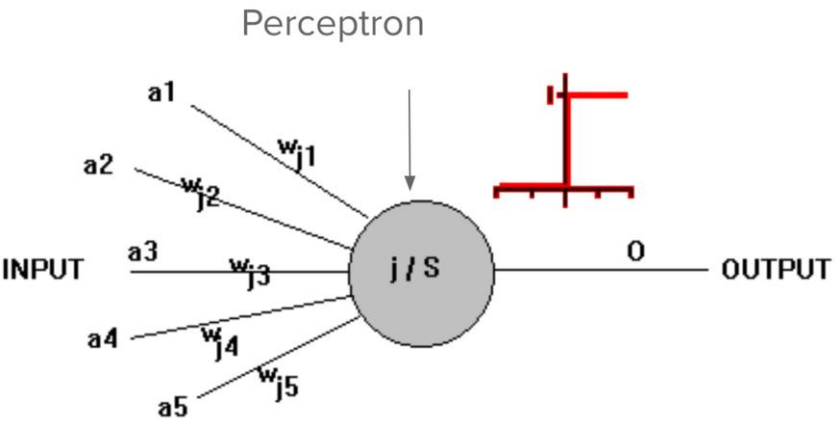
Frank Rosenblatt, using the McCulloch-Pitts neuron & the findings of Hebb, went on to develop the first perceptron

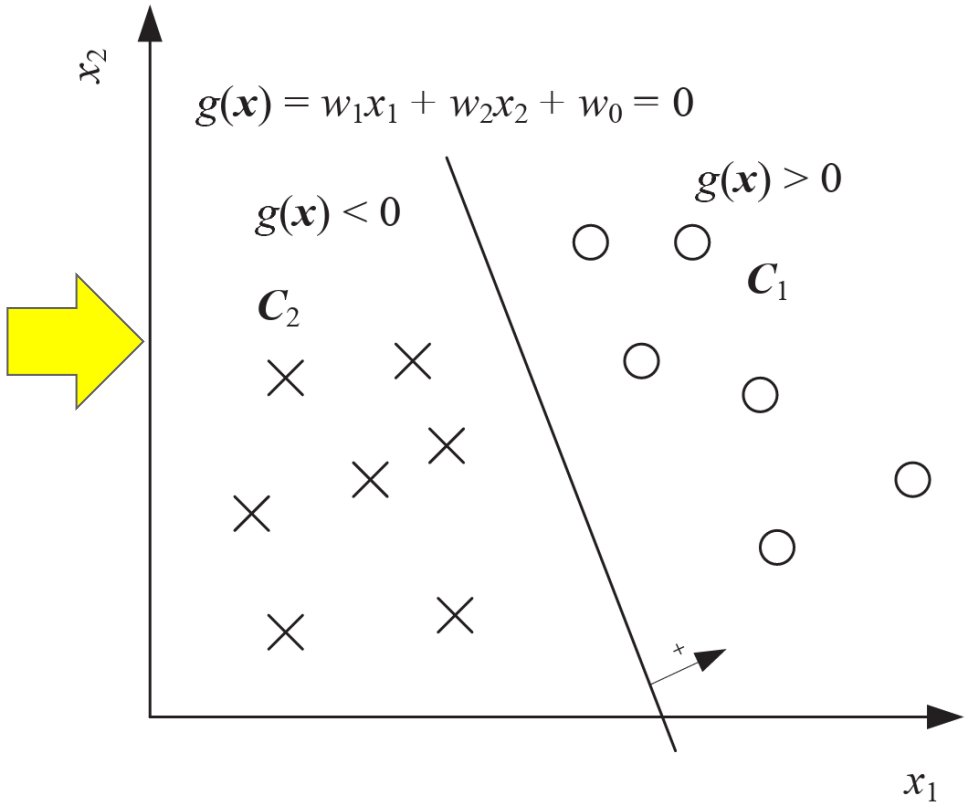


Given an elementary α-perceptron, a stimulus world W, and any classification C(W) for which a solution exists …..

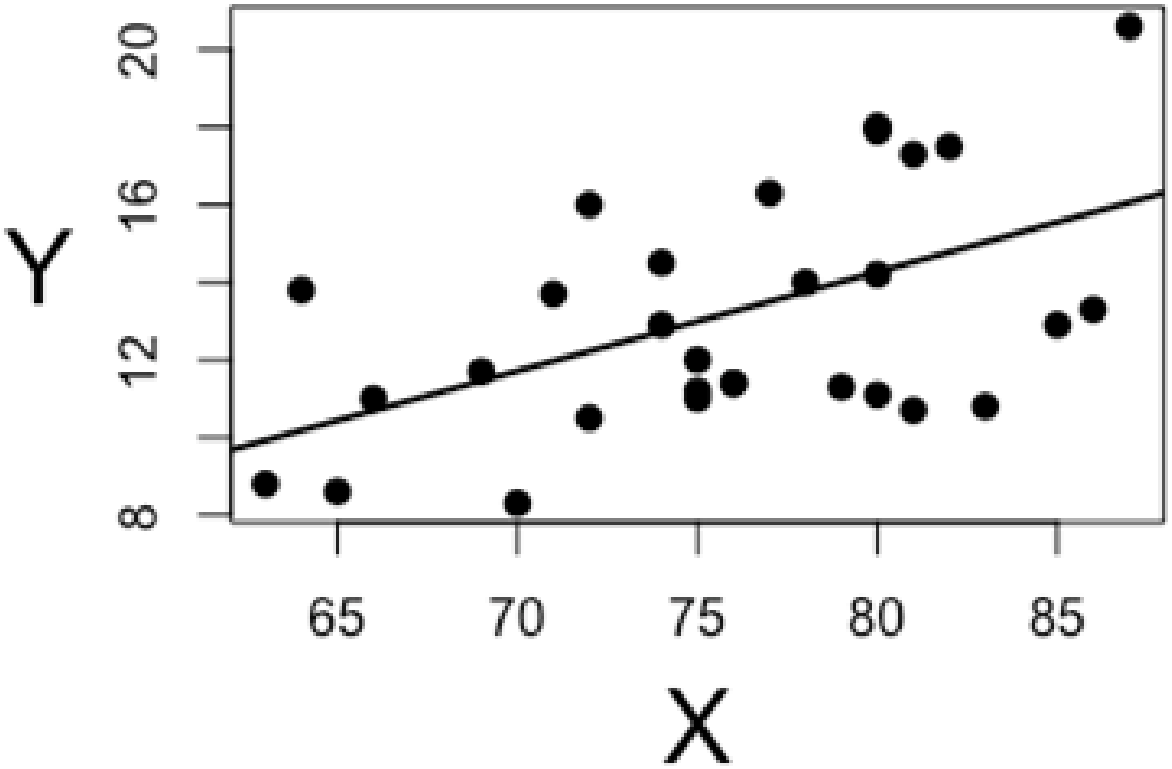
(1958). "The perceptron: A probabilistic model for information storage and organization in the brain"

Geometry of Linear Classifier



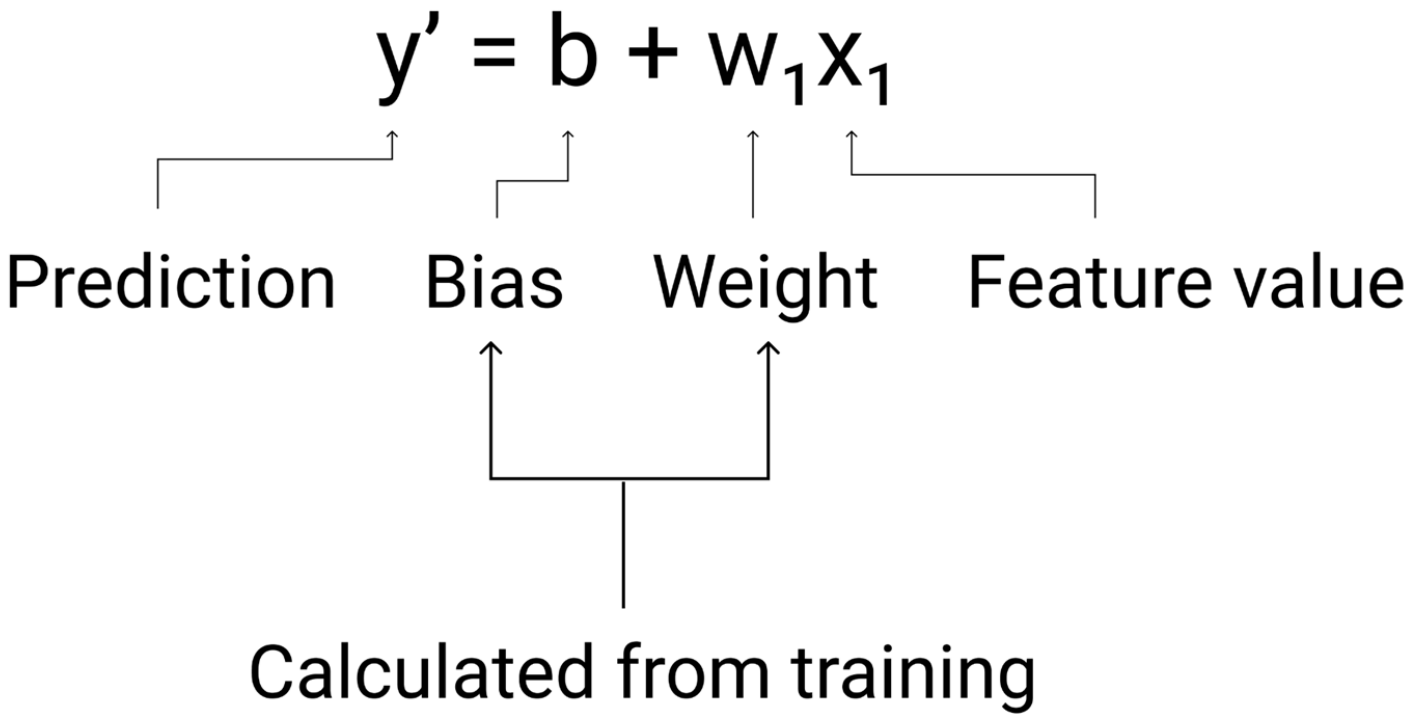


Linear Models with Bias



W.x + b

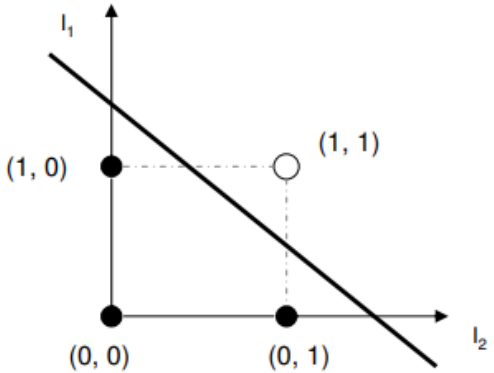
Linear Models with Bias



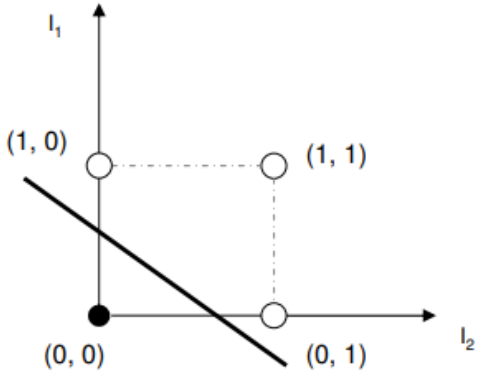
XOR Problem: 1969



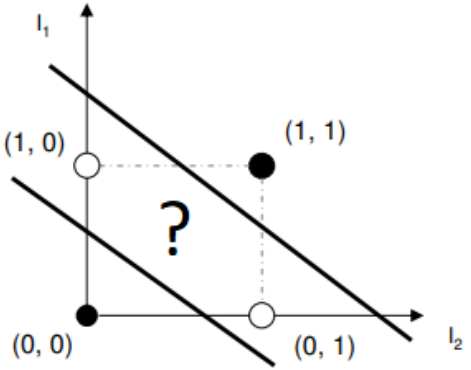
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

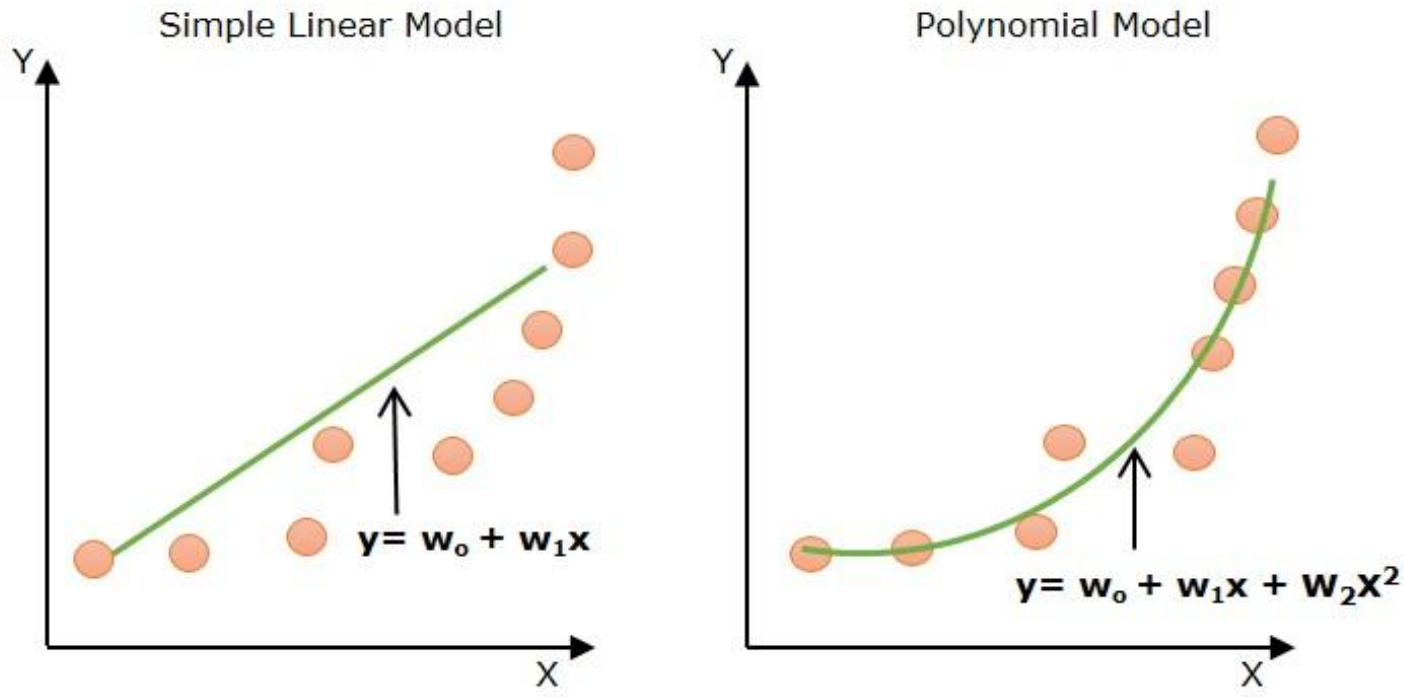


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



1969, Minsky co-authored with Seymour Papert , *Perceptrons: An Introduction to Computational Geometry*.

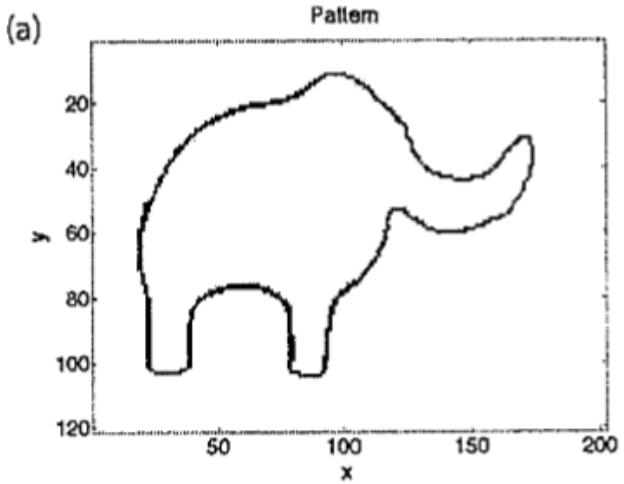
Polynomial Regression



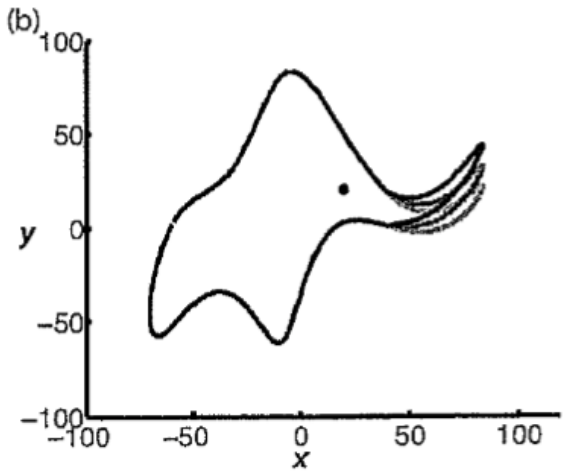
Von Neumann about Fermi's Results

Neumann: How many arbitrary parameters he used for his calculations?
Fermi: Many (with physical interpretation)
Neumann: ***"With four parameters I can fit an elephant, and with five I can make him wiggle his trunk."***

Parameter
$p_1 = 50 - 30i$
$p_2 = 18 + 8i$
$p_3 = 12 - 10i$
$p_4 = -14 - 60i$
$p_5 = 40 + 20i$



Outline of elephant (4 params)



Three snapshots of the wiggling trunk (via 5 th param)

Mayer, Jurgen; Khairy, Khaled; Howard, Jonathon (May 12, 2010). "Drawing an elephant with four complex parameters". *American Journal of Phys*

Polynomial Regression

Ridge Regression: L2 Norm on Weights

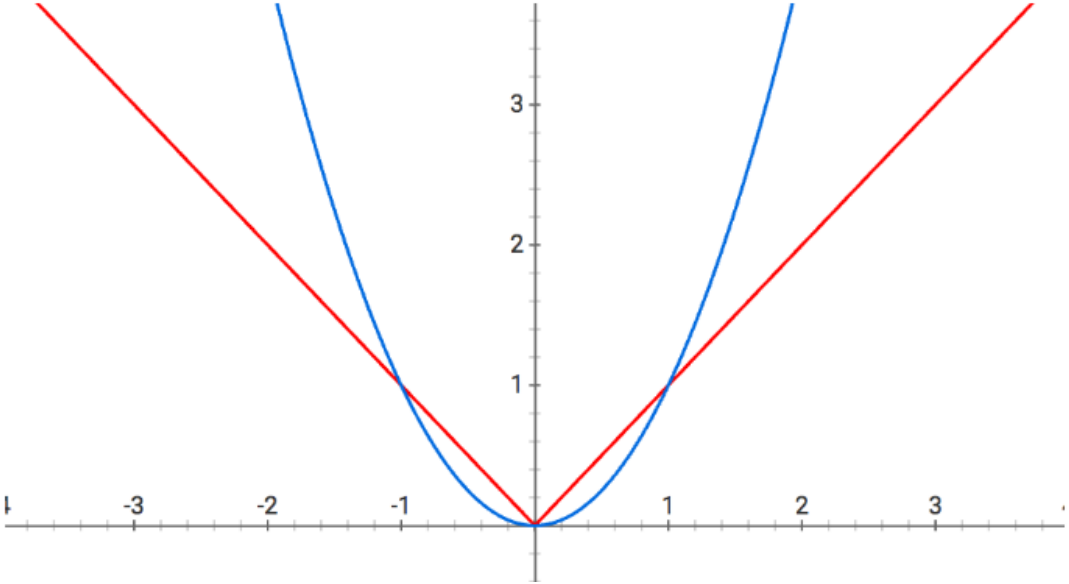
$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^{n} \theta_i^2$$

Lasso Regression: L1 Norm on Weights

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^{n} |\theta_i|$$

Note: (a) θ is the bias and coefficients of the polynomial model
(b) α is the regularization weight

Understanding L1 and L2 Penalty

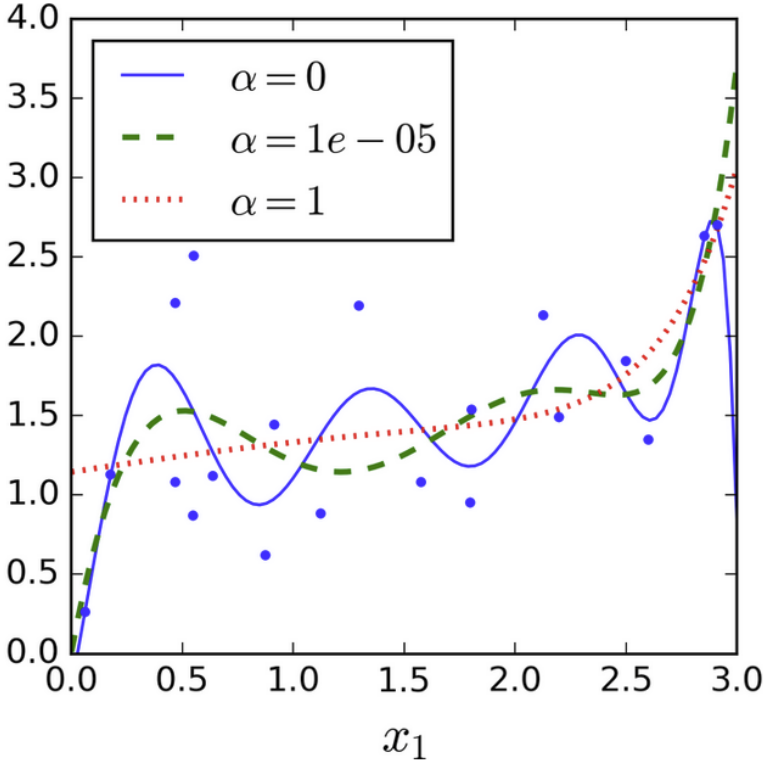


One dimensional case

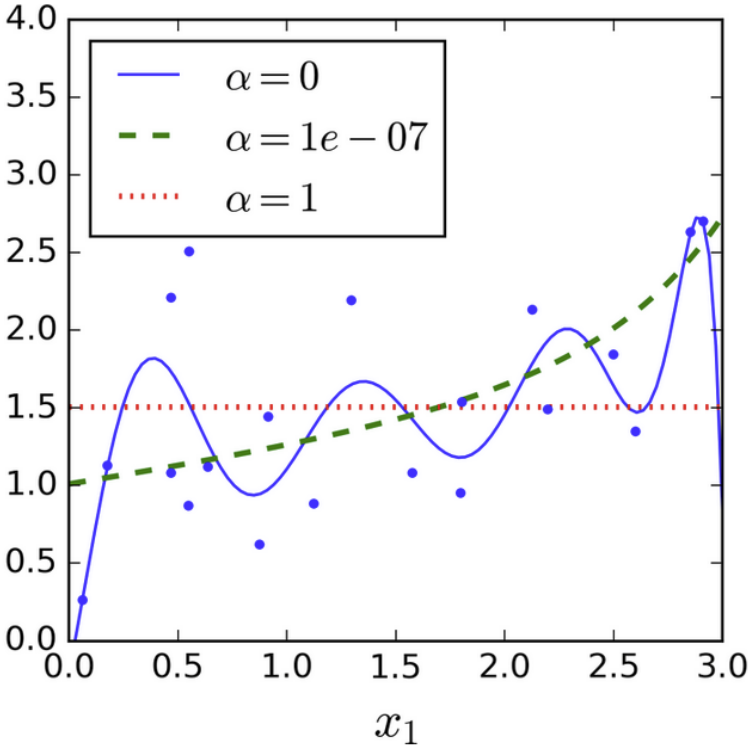
L1: $R(\theta) = \|\theta\|_1 = \sum_{i=1}^{n} |\theta_i|$

L2: $R(\theta) = \|\theta\|_2^2 = \sum_{i=1}^{n} \theta_i^2$

Ridge Vs Lasso Regression

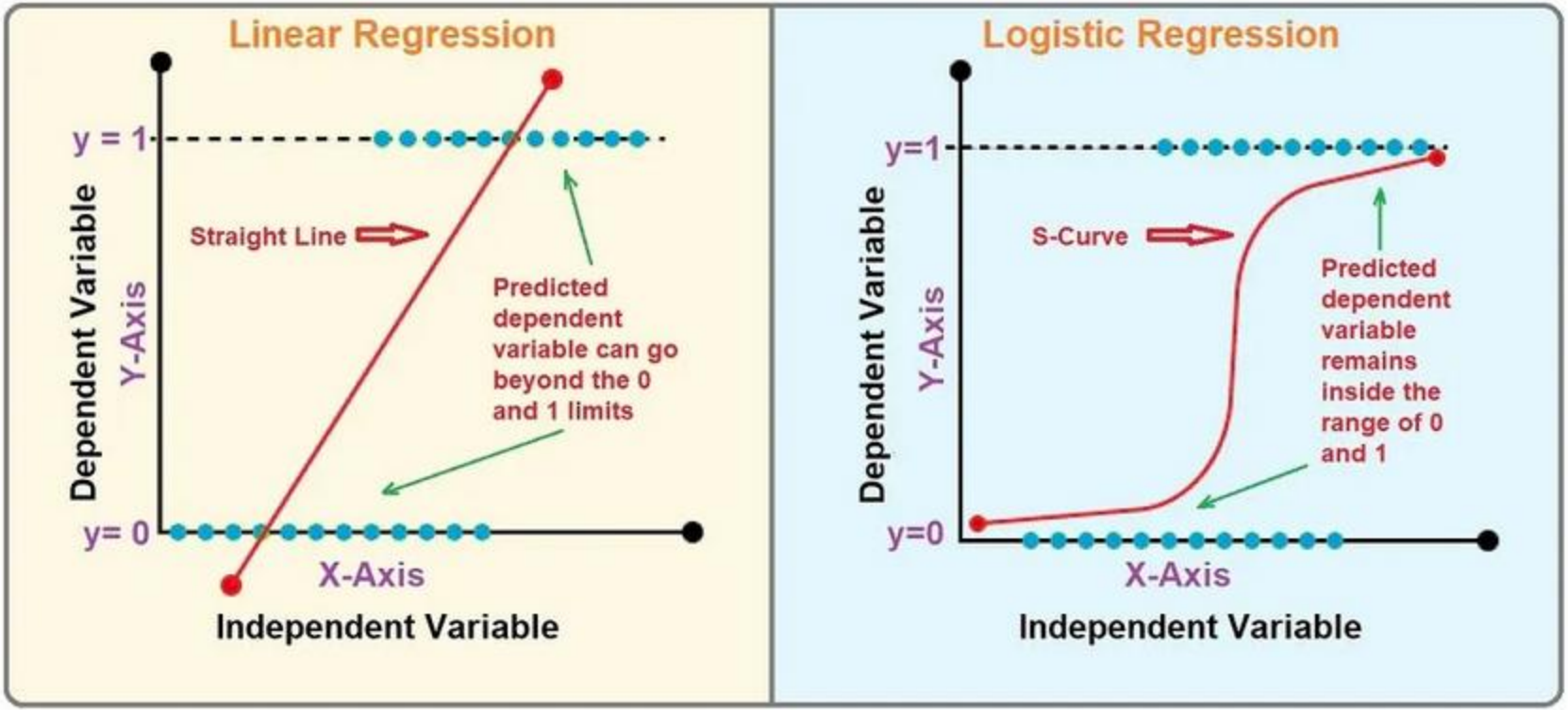


Ridge Regression: L2 Norm on Weights



Lasso Regression: L1 Norm on Weights

Logistic Regression



Logistic Function

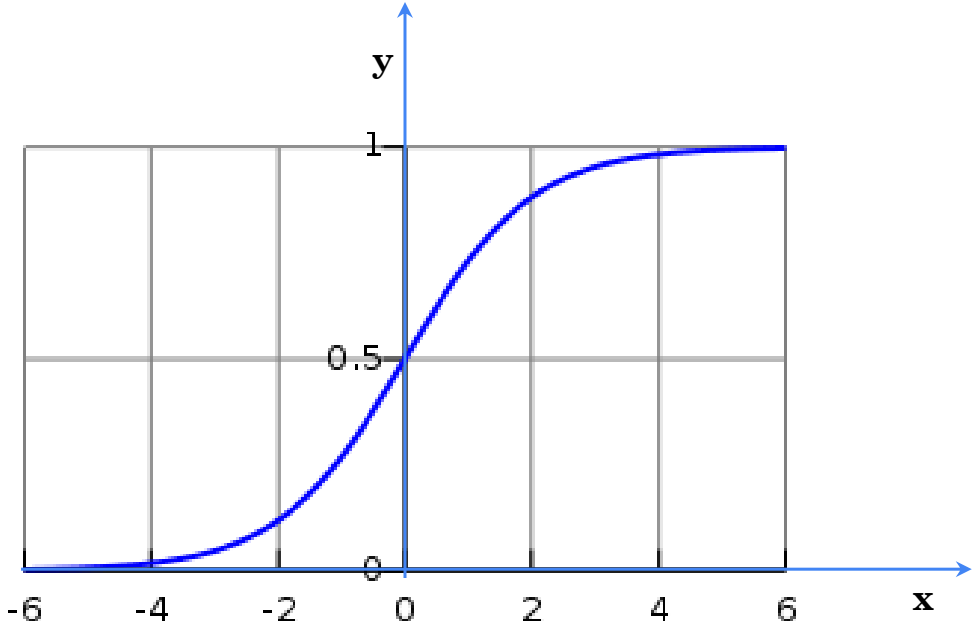
- Mathematical form
  - $\mathrm{LR}(x) = \frac{e^x}{1+e^x}$

Logistic Function

- Mathematical form
  - $\mathrm{LR}(x) = \frac{e^x}{1+e^x}$

- Properties of Logistic function, $\mathrm{LR}(x)$
  1. What is the maximum value?
     close to 1
  2. What is the minimum value?
     close to 0
  3. When it attains maximum value?
     when $x \to \infty$
  4. When it attains minimum value?
     when $x \to -\infty$

Shape of Logistic Function



Other names

- **Sigmoid function**
- **S-curve (because of S-like shape)**

Preliminaries

- **Inherently a 2-class classifier**
  - The classes traditionally called
    - 0 and 1

    OR
    - positive and negative

- **Logistic Regression is a probabilistic classifier**
  - Given an input feature vector $\boldsymbol{x}$, it computes the probability that $\boldsymbol{x}$ belongs to class 1 (or positive)

$$\boldsymbol{P(y = 1|x)}$$

Preliminaries

"odds" measure the likelihood of an event by comparing the probability of it happening to the probability of it not happening (Odds = P/(1-P))

- Odds

$$\frac{P(y=1|x)}{1-P(y=1|x)}$$

- Properties of odds
  1. Always positive
  2. Lies between 0 and ∞

Logistic Regression Model



- **Main assumption**
  - The logarithm of odds is a linear function of the features of input x

- **Mathematically**

  [assume that x has n features: $\mathbf{x} = (x_1, x_2, \ldots\ldots, x_n)$]

$$log\left(\frac{P(y=1|x)}{1-P(y=1|x)}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots\ldots + \theta_n x_n = Z\ (say)$$

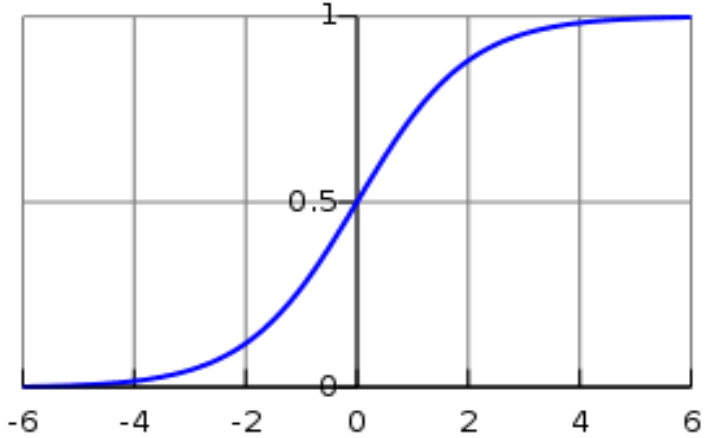
  Organizing the above you get

$$\mathbf{P(y=1|x)} = \frac{e^Z}{1+e^Z}$$

- **For convenience we write, instead**

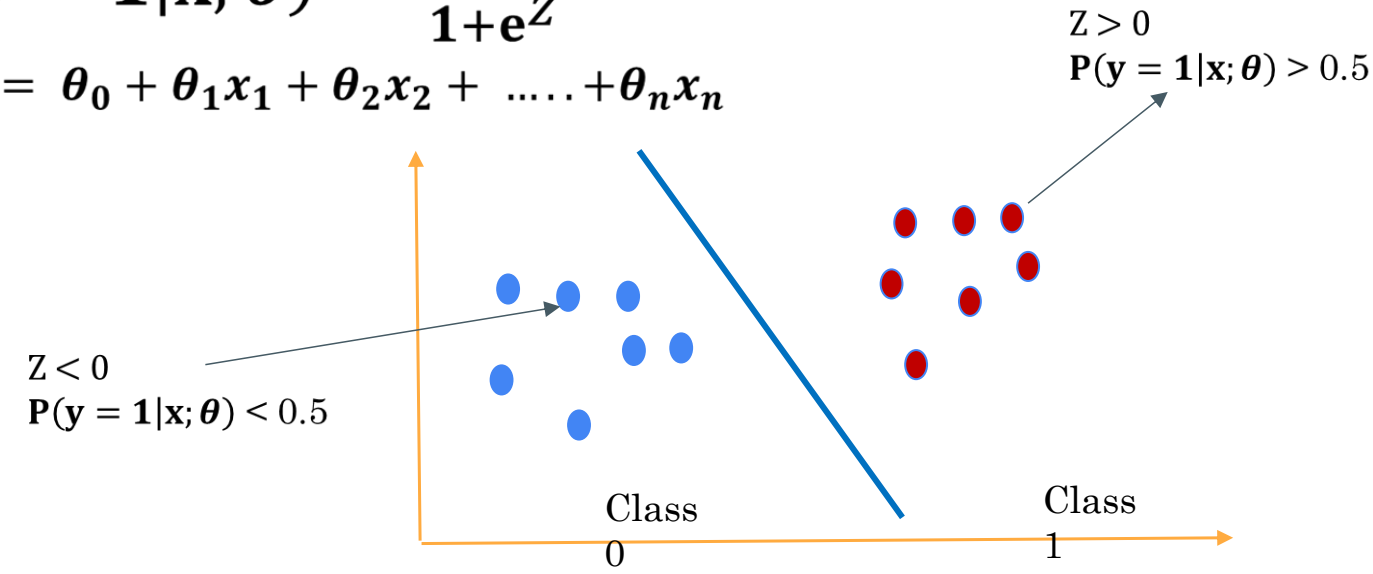
$$\mathbf{P(y=1|x;\theta)} = \frac{e^Z}{1+e^Z}$$

where θ is a short form of model parameters $(\theta_0, \theta_1, \ldots. \theta_n)$



What kind of boundary it can model?

- $\mathbf{P(y = 1|x; \theta)} = \dfrac{e^{Z}}{1+e^{Z}}$
  - $Z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots\ldots + \theta_n x_n$

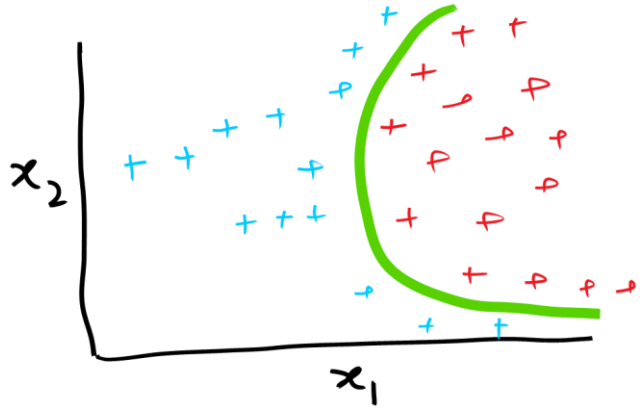


What about non-linear boundary?



Dealing with Non-linear Boundary

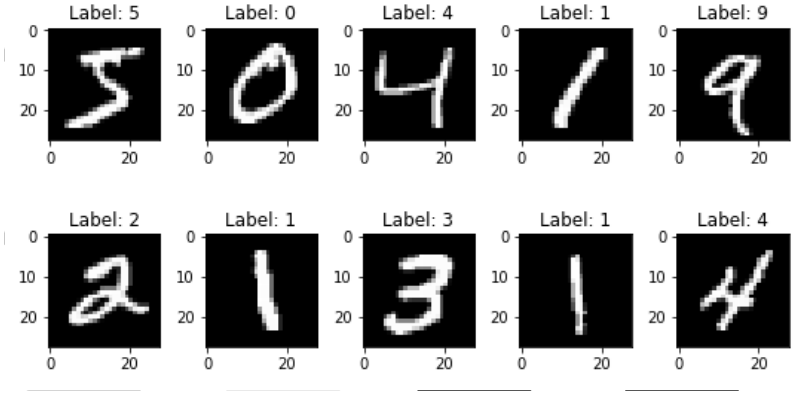
Z determines the boundary



In this case, Z needs to be a quadratic function

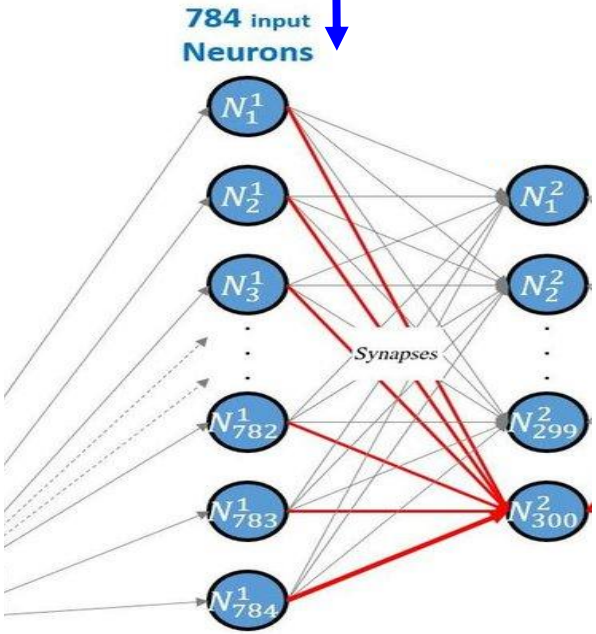
Recap: AI: How to Solve it?

1. Collect Labelled Dataset ✓
2. Design ANN Architecture ✓
3. Define Loss Function ✓
4. Optimize weights

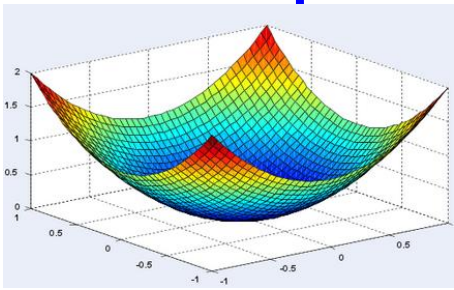


1. Collect Labelled dataset

4. Optimize Weight



2. Design Artificial Neural Network



3. Loss Function

Optimize Weights

Review of Differential Calculus

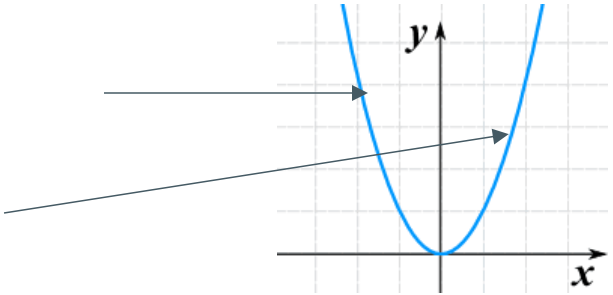
- **Consider the univariate function**

  y = f(x) (for example: $y = x^2$)

- $\frac{dy}{dx}$ **at x=x_0 means rate of change of y w.r.t x**
  - $\frac{dy}{dx}$ < 0 means the function is decreasing
  - $\frac{dy}{dx}$ > 0 means the function is increasing

What is the sign of $\frac{dy}{dx}$ here?

What is the sign of $\frac{dy}{dx}$ here?

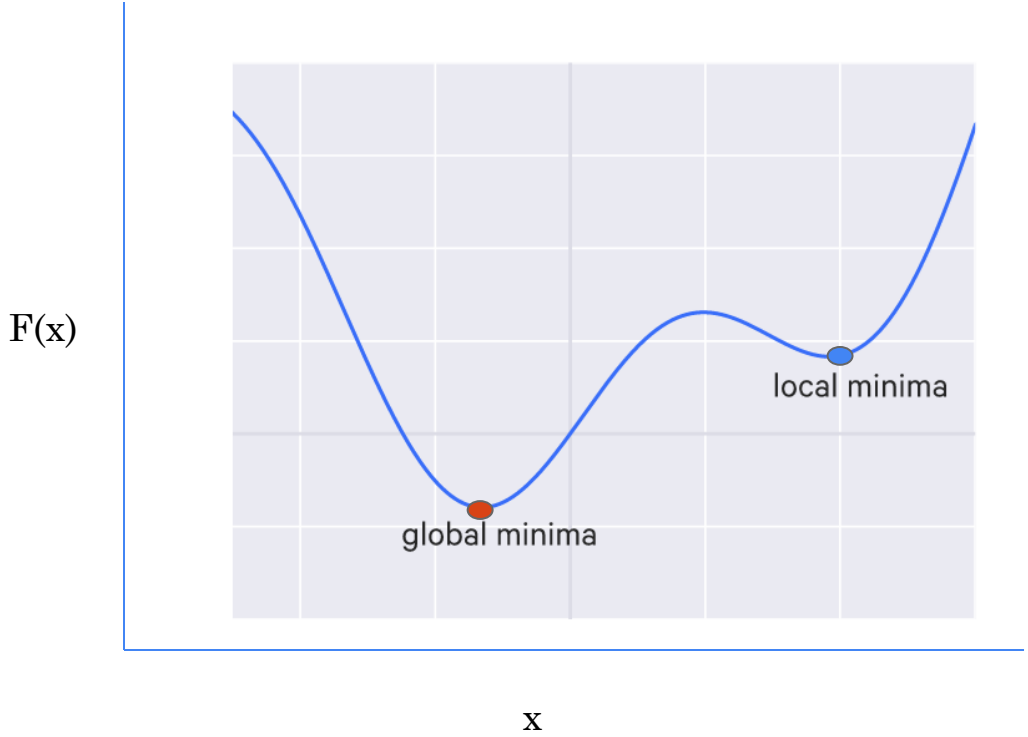


Review of Differential Calculus

- Multivariate function
  - Function containing more than one independent variable
  - Example: $y = x^2 + z^2 + w^2$

- Partial differentiation
  - Vary one variable at a time and keep others fixed
    - $\frac{\partial y}{\partial x} = 2x$, $\frac{\partial y}{\partial z} = 2z$, $\frac{\partial y}{\partial w} = 2w$

Local and Global minima



Optimize Weights via Loss Function

- **Goal**

  Minimize a function w.r.t the parameters that the function contains

- **The error/objective function**

$$E = \sum_{i=1}^{m} \left(y^{(i)} - F(x^{(i)})\right)^2$$

$$F \rightarrow ML\ model$$

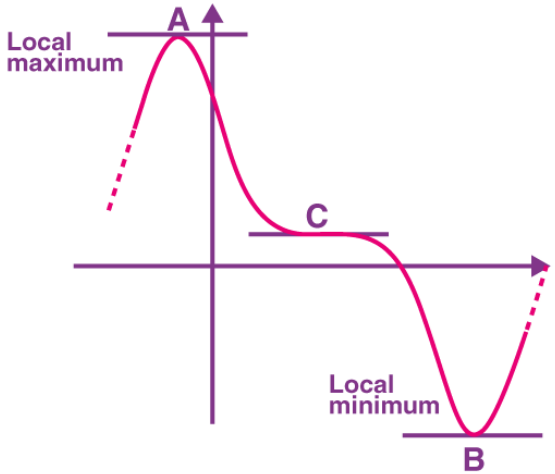
Training data	
Input	Output

- **Two objectives**

1. Only minimum value of the function
2. Find the value of the parameters that will minimize the function

Closed Form Expression

	Maximum	Minimum
Necessary condition	$\frac{dy}{dx} = 0$	$\frac{dy}{dx} = 0$
Sufficient condition	$\frac{dy}{dx} = 0 \; ; \; \frac{d^2y}{dx^2} < 0$	$\frac{dy}{dx} = 0; \; \frac{d^2y}{dx^2} > 0$



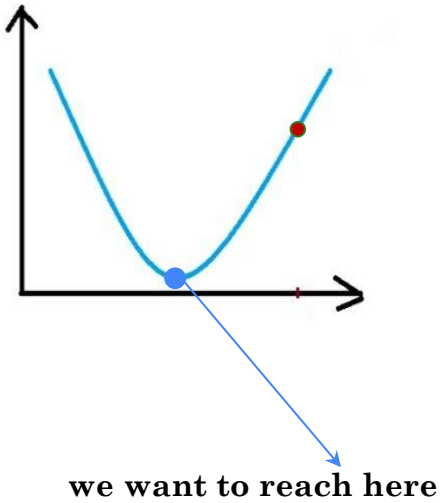
Gradient descent



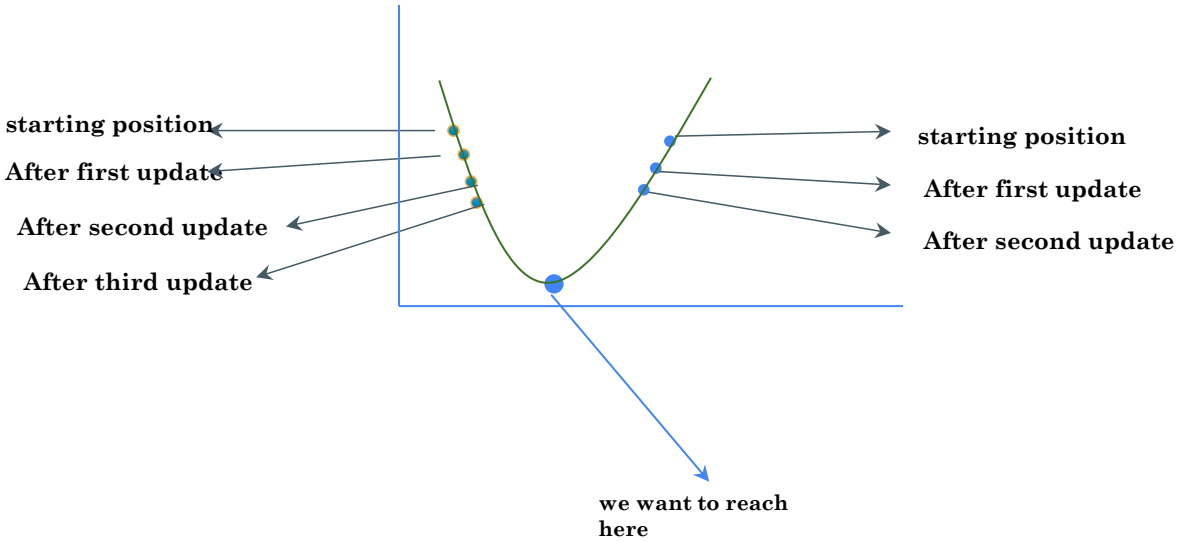


Gradient Descent: Demonstration

- **Role of gradient descent**
  - To find the values of parameters that minimize an objective function

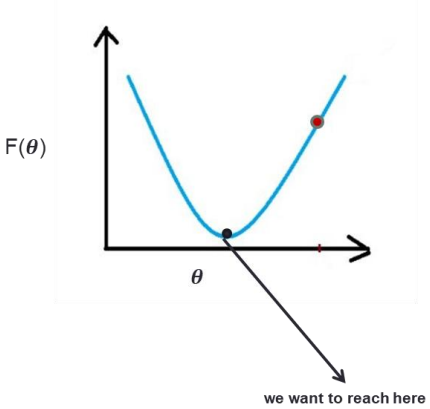


Gradient Descent: Demonstration



Gradient Descent Algorithm

1. **Randomly set the values of parameters (thetas)**
2. **Repeat until convergence**
   - $\theta_j^{t+1} = \theta_j^t - r * \frac{\partial E}{\partial \theta_j}$ **for all j**



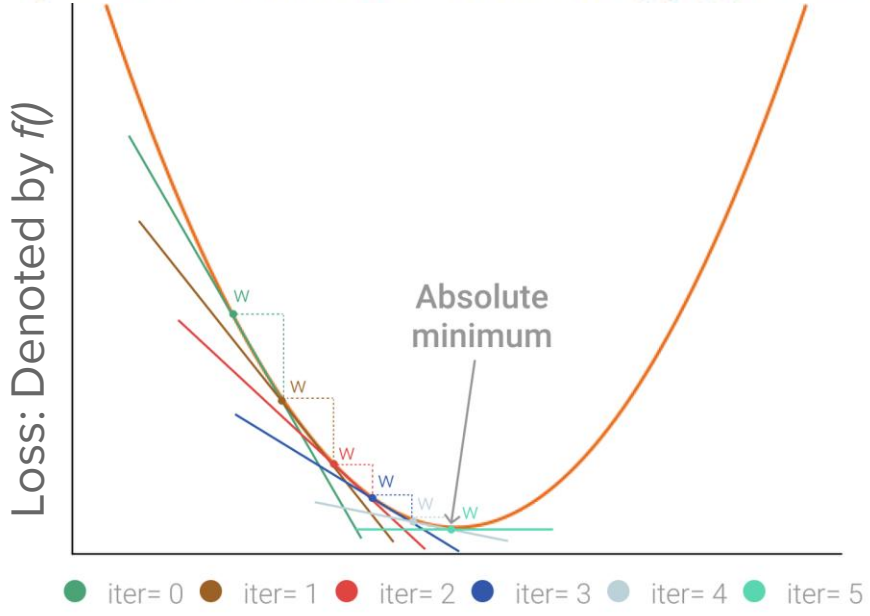
- **How to check convergence?**
  - **Two widely used approaches**
    1. Iterate a fixed number of times
    2. Stop if parameters do not change much in two consecutive iteration

Gradient Descent

Definition 1 *Suppose f is a real valued function and a is a point in its domain of definition. The derivative of f at a is defined by*

$$\lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

provided this limit exists. Derivative of $f(x)$ at a is denoted by $f'(a)$.

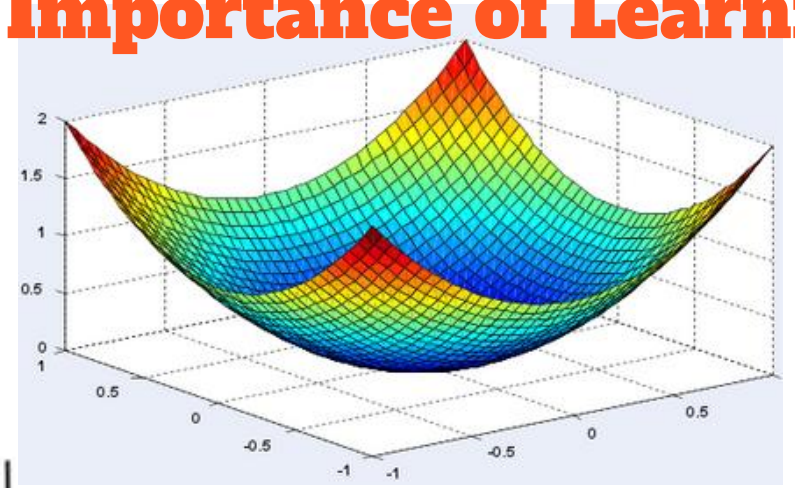


What happens to Loss *f(a+h) - f(a)* if the weight update *h* is $-\lambda f'(a)$?
Note: $\lambda > 0$

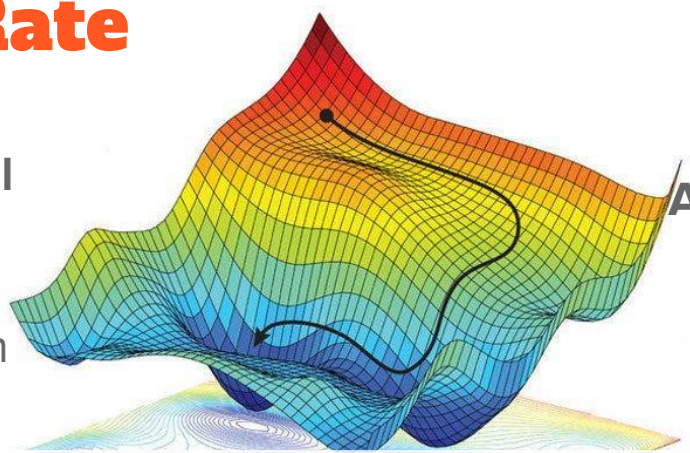
Answer: f(a+h) - f(a) is negative, which means f(a+h) ≤ f(a).

Optimizing: Update each weight a as $a - \lambda f'(a)$, where λ is the learning rate.

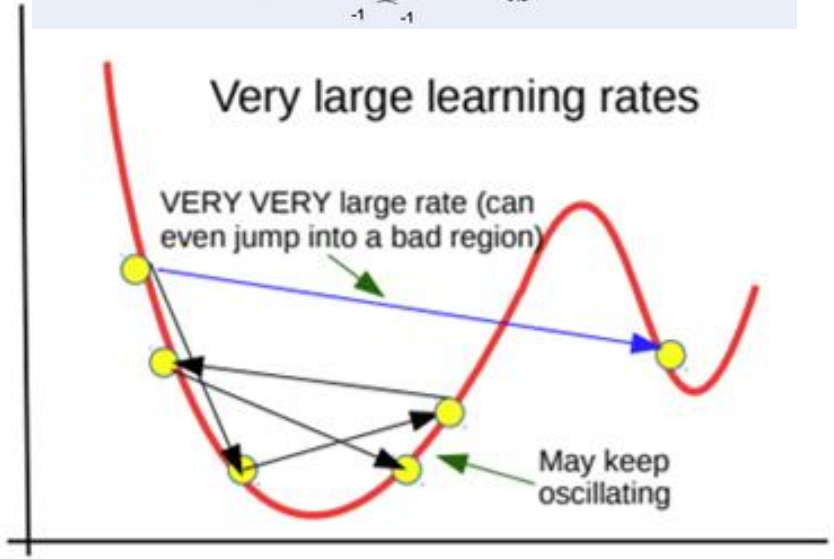
Importance of Learning Rate

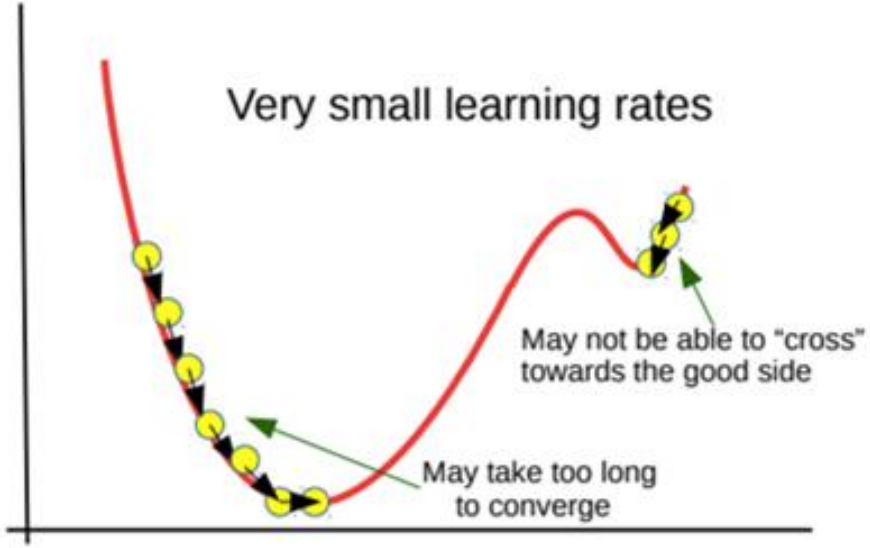


An Ideal Case of Loss function



A Practical Case of Loss function





Gradient Descent for Linear Regression

$$y = F(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots\ldots + \theta_n x_n$$

- **Error function**

  $E = \sum_{i=1}^{m} \left(F\left(x^{(i)}\right) - y^{(i)}\right)^2$ (m = no. of training samples)

- **Gradient descent update rule**
  - Requires partial differentiation with respect to all parameters

  $\theta_i^{t+1} = \theta_i^{t} - r * \frac{\partial E}{\partial \theta_i}$ for all i

Gradient Descent for Linear Regression

- $E_t = \sum_{i=1}^{m} \left(F\left(x^{(i)}\right) - y^{(i)}\right)^2$ (m = # training sample)

  For example: $F(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots\ldots + \theta_n x_n$

- $\frac{\partial E_t}{\partial \theta_0} = 2 \sum_{i=1}^{m} (F\left(x^{(i)}\right) - y^{(i)})$
- $\frac{\partial E_t}{\partial \theta_1} = 2 \sum_{i=1}^{m} (F\left(x^{(i)}\right) - y^{(i)}) * \mathbf{x_1^i}$

- So, in general

  - $\frac{\partial E_t}{\partial \theta_j} = 2 \sum_{i=1}^{m} (F\left(x^{(i)}\right) - y^{(i)}) * \mathbf{x_j^i}$ (for all j) (assume $\mathbf{x_0^i} = 1$)

- Update rule:

  $\theta_j = \theta_j - r * 2 \sum_{i=1}^{m} (F\left(x^{(i)}\right) - y^{(i)}) * \mathbf{x_j^i}$ (for all j)

-

Gradient Descent for Logistic Regression

- **Take one sample from the training data**
  - Let $\boldsymbol{x}$ be the input sample/feature vector
  - Let $\boldsymbol{y}$ be its label/class

- **Given, the input sample x and the model parameters $\boldsymbol{\theta}$**

  - **What is the probability that x belongs to class 1 ?**
    - $\boldsymbol{P(y = 1|x; \theta) = LR(x)}$

  - **What is the probability that x belongs to class 0 ?**
    - $\boldsymbol{P(y = 0|x; \theta) = 1 - LR(x)}$

- **Putting together**

$$\boldsymbol{P(y|x; \theta) = (LR(x))^{y} \times \big(1 - LR(x)\big)^{1-y}}$$

Gradient Descent for Logistic Regression

- Likelihood function (We have n training examples)
  - $L(\theta) = p(Y|X;\theta)$, $where\ Y = (y^1, y^2, \ldots, y^n)$ $and\ X = (x^1, x^2, \ldots, x^n)$
- Assume that training examples are independent
  - $L(\theta) = p(Y|X;\theta) = \prod_{i=1}^{n} p(y^i|x^i;\theta)$

$$= \prod_{i=1}^{n} LR(x^i)^{y^i} \times \left(1 - LR(x^i)\right)^{1-y^i}$$

Apply: $P(y|x;\theta) = LR(x)^y \times \left(1 - LR(x)\right)^{1-y}$

Estimating Model Parameters

- We now have the likelihood function:

$$L(\theta) = \prod_{i=1}^{n} LR(x^i)^{y^i} \times \left(1 - LR(x^i)\right)^{1-y^i}$$

- What we have to do now?

  Maximize the likelihood function: $L(\theta) = \prod_{i=1}^{n} LR(x^i)^{y^i} \times \left(1 - LR(x^i)\right)^{1-y^i}$

- But $L(\theta)$ is complex to manage

- Alternative solution
  - Maximize $\log L(\theta)$ instead

- So, the final likelihood function is

$$\log L(\theta) = \sum_{i=1}^{n} y^i \log LR(x^i) + \left(1 - y^i\right) \log \left(1 - LR(x^i)\right)$$

Estimating Model Parameters

- **Apply stochastic gradient ascent**
  - Updating parameters using one training sample at a time
- **Gradient ascent:** $\theta_j^{new} = \theta_j^{old} + r \frac{\partial log(L(\theta))}{\partial \theta_j}$
- **We just need to find out** $\frac{\partial log(L(\theta))}{\partial \theta_j}$, for all j
- **Likelihood function:**

$$\log L(\theta) = y^i log\, LR(x^i) + (1 - y^i) log(1 - LR(x^i))$$

- $\frac{\partial log(L(\theta))}{\partial \theta_j} = \left(y^i - LR(x^i)\right) * x_j^i$

x_j^i is the j-th feature of i-th training sample